

LAPORAN PROJEK TUGAS AKHIR
MATA KULIAH PEMROGRAMAN BASIS DATA IMPLEMENTASI SISTEM
BASIS DATA INVENTARIS LABORATORIUM MENGGUNAKAN STORED
PROCEDURE, FUNCTION, TRIGGER, TRANSACTION CONTROL, CUR-
SOR, DAN INDEXING



Disusun oleh Kelompok 7:

Fadhila suardi	IK2411018
Nur fahila dwi irfani	IK2411031
Nur octavia kaila ramadhani	IK2411038
Riadarmawangsi	IK2411056

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MEGA BUANA PALOPO
2024/2025

BAB I PENDAHULUAN

1.1 Latar Belakang

Laboratorium merupakan sarana penunjang kegiatan praktikum dan penelitian yang memiliki jumlah peralatan dan bahan yang cukup banyak serta terus berubah kondisinya dari waktu ke waktu. Pengelolaan inventaris laboratorium yang masih dilakukan secara manual atau menggunakan pencatatan sederhana rentan menimbulkan permasalahan, seperti data stok yang tidak akurat, sulitnya pelacakan riwayat peminjaman, serta tidak adanya jejak audit ketika terjadi perubahan data barang.

Berdasarkan permasalahan tersebut, dibutuhkan sebuah sistem basis data yang tidak hanya menyimpan data secara terstruktur, tetapi juga mampu menerapkan logika bisnis langsung pada sisi basis data melalui Stored Procedure, Function, Trigger, Transaction Control, Exception Handling, Cursor, dan Indexing. Pendekatan ini dipilih karena mampu menjaga konsistensi data secara otomatis tanpa sepenuhnya bergantung pada aplikasi di sisi klien.

Projek ini mengangkat tema Sistem Basis Data Inventaris Laboratorium sebagai studi kasus untuk mengintegrasikan seluruh materi Pemrograman Basis Data yang telah dipelajari, mulai dari perancangan skema, penerapan constraint, hingga optimasi query menggunakan indexing.

1.2 Rumusan Masalah

1. Bagaimana merancang skema basis data inventaris laboratorium yang mampu merepresentasikan proses peminjaman, pengembalian, dan pengelolaan stok barang secara akurat?
2. Bagaimana menerapkan Stored Procedure dan Function untuk mengotomatisasi proses bisnis utama seperti peminjaman, pengembalian, dan perhitungan denda?
3. Bagaimana menerapkan Trigger untuk validasi data otomatis dan pencatatan audit log setiap terjadi perubahan data penting?
4. Bagaimana menerapkan Transaction Control dan Exception Handling agar transaksi tetap konsisten meskipun terjadi kegagalan di tengah proses?
5. Bagaimana penerapan Indexing dapat meningkatkan efisiensi query pada tabel yang berukuran besar?

1.3 Tujuan Projek

1. Merancang skema basis data inventaris laboratorium beserta relasi, primary key, foreign key, dan constraint-nya.
2. Membuat Stored Procedure untuk proses peminjaman, pengembalian, pembatalan transaksi, pembaruan status keterlambatan, dan pelaporan.
3. Membuat Function untuk perhitungan denda, pengecekan stok, dan jumlah peminjaman aktif.
4. Membuat Trigger untuk validasi stok, audit logging, dan pembaruan stok secara otomatis.
5. Menerapkan Transaction Control (COMMIT, ROLLBACK, SAVEPOINT) dan Exception Handling pada proses transaksi.
6. Menerapkan Cursor untuk memproses data lebih dari satu baris.
7. Menerapkan Indexing dan membandingkan performa query sebelum dan sesudah index dibuat.

1.4 Manfaat Projek

- Bagi mahasiswa: memperdalam pemahaman mengenai pemrograman basis data pada studi kasus nyata.
- Bagi laboratorium: memberikan gambaran rancangan sistem yang dapat mendukung pengelolaan inventaris secara lebih tertib dan akuntabel.
- Bagi pengembangan sistem selanjutnya: menjadi dasar rancangan basis data yang dapat diintegrasikan dengan aplikasi front-end pada projek berikutnya.

1.5 Ruang Lingkup Projek

Projek ini berfokus pada perancangan dan implementasi basis data menggunakan MySQL/MariaDB (XAMPP), meliputi pembuatan tabel dan relasi, Stored Procedure, Function, Trigger, Transaction Control, Exception Handling, Cursor, dan Indexing. Projek ini tidak mencakup pembuatan aplikasi antarmuka (front-end) secara lengkap; pengujian dilakukan melalui query dan pemanggilan prosedur/fungsi langsung pada DBMS.

BAB II ANALISIS DAN PERANCANGAN DATABASE

2.1 Deskripsi Studi Kasus

Laboratorium pada suatu institusi pendidikan memiliki berbagai peralatan dan bahan praktikum yang dikelola oleh petugas laboran. Peralatan tersebut dipinjam oleh mahasiswa, dosen, maupun staf untuk keperluan praktikum atau penelitian, kemudian dikembalikan setelah selesai digunakan. Selama ini pencatatan peminjaman, stok, dan kondisi barang dilakukan secara manual sehingga rawan terjadi selisih data, keterlambatan pengembalian yang tidak terpantau, dan tidak adanya catatan riwayat perubahan data barang.

Sistem Basis Data Inventaris Laboratorium dirancang untuk menangani empat proses utama, yaitu: pengelolaan data barang dan kategori, pencatatan peminjaman oleh peminjam, pencatatan pengembalian beserta perhitungan denda keterlambatan, dan pencatatan audit atas perubahan data penting.

2.2 Analisis Kebutuhan Data

Berdasarkan studi kasus di atas, entitas data yang dibutuhkan meliputi:

- Data kategori barang, untuk mengelompokkan jenis peralatan/bahan laboratorium.
- Data laboratorium, sebagai lokasi penempatan barang.
- Data petugas (laboran), sebagai pihak yang memproses transaksi.
- Data peminjam (mahasiswa/dosen/staf), sebagai pihak yang meminjam barang.
- Data barang, mencakup stok total, stok tersedia, kondisi, dan harga.
- Data transaksi peminjaman beserta detail barang yang dipinjam.
- Data pengembalian barang beserta denda keterlambatan.
- Data audit log untuk mencatat riwayat perubahan data penting.

2.3 Analisis Proses Bisnis

1. Proses Peminjaman

Peminjam mengajukan peminjaman barang kepada petugas. Sistem memeriksa ketersediaan stok barang. Apabila stok mencukupi, transaksi peminjaman disimpan dan stok barang otomatis berkurang. Apabila stok tidak mencukupi, seluruh proses dibatalkan (ROLLBACK) dan sistem memberikan pesan kesalahan kepada petugas.

2. Proses Pengembalian

Peminjam mengembalikan barang kepada petugas. Sistem menghitung denda apabila pengembalian melewati tanggal rencana kembali, mencatat kondisi barang saat dikembalikan, mengembalikan (menambah) stok barang secara otomatis, dan memperbarui status transaksi peminjaman menjadi 'dikembalikan'.

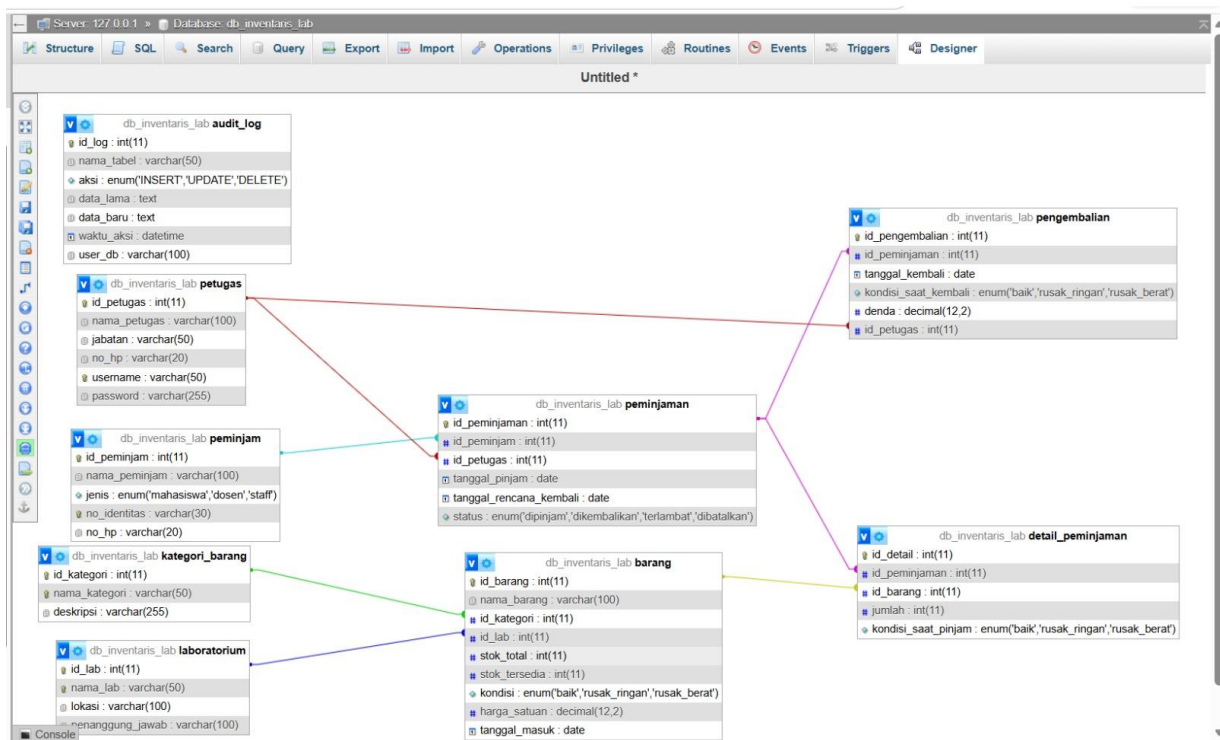
3. Proses Pembatalan Transaksi

Apabila peminjaman dibatalkan sebelum barang diambil/digunakan, sistem mengembalikan stok barang untuk setiap item pada transaksi tersebut menggunakan cursor, serta menandai status transaksi menjadi 'dibatalkan'.

4. Proses Audit Logging

Setiap terjadi perubahan (INSERT/UPDATE) pada data penting seperti tabel barang dan peminjaman, sistem secara otomatis mencatat data lama dan data baru ke dalam tabel audit_log melalui trigger, tanpa perlu ditangani secara manual oleh aplikasi.

2.4 Entity Relationship Diagram (ERD)



2.5 Struktur Tabel

1. Tabel kategori_barang

Nama Field	Tipe Data	Constraint	Keterangan
id_kategori	INT	PK, AUTO_INCREMENT	Identitas kategori barang
nama_kategori	VAR-CHAR(50)	NOT NULL, UNIQUE	Nama kategori barang
deskripsi	VAR-CHAR(255)	-	Keterangan kategori

2. Tabel laboratorium

Nama Field	Tipe Data	Constraint	Keterangan
id_lab	INT	PK, AUTO_INCREMENT	Identitas laboratorium
nama_lab	VAR-CHAR(50)	NOT NULL, UNIQUE	Nama laboratorium
lokasi	VAR-CHAR(100)	NOT NULL	Lokasi laboratorium
penanggung_jawab	VAR-CHAR(100)	NOT NULL	Penanggung jawab laboratorium

3 Tabel petugas

Nama Field	Tipe Data	Constraint	Keterangan
id_petugas	INT	PK, AUTO_INCREMENT	Identitas petugas
nama_petugas	VAR-CHAR(100)	NOT NULL	Nama petugas/laboran
jabatan	VAR-CHAR(50)	NOT NULL	Jabatan petugas
no_hp	VAR-CHAR(20)	-	Nomor telepon
username	VAR-CHAR(50)	NOT NULL, UNIQUE	Username login
password	VAR-CHAR(255)	NOT NULL	Kata sandi (hash)

4 Tabel peminjam

Nama Field	Tipe Data	Constraint	Keterangan
id_peminjam	INT	PK, AUTO_INCREMENT	Identitas peminjam
nama_peminjam	VAR-CHAR(100)	NOT NULL	Nama peminjam
jenis	ENUM	NOT NULL, DEFAULT 'mahasiswa'	mahasiswa / dosen / staff
no_identitas	VAR-CHAR(30)	NOT NULL, UNIQUE	NIM/NIP/identitas lain
no_hp	VAR-CHAR(20)	-	Nomor telepon

5 Tabel barang

Nama Field	Tipe Data	Constraint	Keterangan
id_barang	INT	PK, AUTO_INCREMENT	Identitas barang
nama_barang	VAR-CHAR(100)	NOT NULL	Nama barang/peralatan
id_kategori	INT	FK -> kategori_barang	Kategori barang
id_lab	INT	FK -> laboratorium	Lokasi laboratorium barang
stok_total	INT	NOT NULL, CHECK >= 0	Jumlah total barang
stok_tersedia	INT	NOT NULL, CHECK >= 0, <= stok_total	Jumlah stok siap dipinjam

Nama Field	Tipe Data	Constraint	Keterangan
kondisi	ENUM	NOT NULL, DEFAULT 'baik'	baik / rusak_ringan / rusak_berat
harga_satuan	DECIMAL(12,2)	NOT NULL, CHECK >= 0	Harga per unit
tanggal_masuk	DATE	NOT NULL, DEFAULT CURRENT_DATE	Tanggal barang masuk inventaris

6 Tabel peminjaman

Nama Field	Tipe Data	Constraint	Keterangan
id_peminjaman	INT	PK, AUTO_INCREMENT	Identitas transaksi peminjaman
id_peminjam	INT	FK -> peminjam	Peminjam barang
id_petugas	INT	FK -> petugas	Petugas yang memproses
tanggal_pinjam	DATE	NOT NULL, DEFAULT CURRENT_DATE	Tanggal peminjaman
tanggal_rencana_kembali	DATE	NOT NULL, CHECK >= tanggal_pinjam	Batas waktu pengembalian
status	ENUM	NOT NULL, DEFAULT 'dipinjam'	dipinjam/dikembalikan/terlambat/dibatalkan

7. Tabel detail_peminjaman

Nama Field	Tipe Data	Constraint	Keterangan
id_detail	INT	PK, AUTO_INCREMENT	Identitas detail peminjaman
id_peminjaman	INT	FK -> peminjaman (ON DELETE CASCADE)	Transaksi peminjaman terkait
id_barang	INT	FK -> barang	Barang yang dipinjam
jumlah	INT	NOT NULL, CHECK > 0	Jumlah unit dipinjam
kondisi_saar_pinjam	ENUM	NOT NULL, DEFAULT 'baik'	Kondisi barang saat dipinjam

8. Tabel pengembalian

Nama Field	Tipe Data	Constraint	Keterangan
id_pengembalian	INT	PK, AUTO_INCREMENT	Identitas pengembalian
id_peminjaman	INT	FK -> peminjaman	Transaksi peminjaman terkait
tanggal_kembali	DATE	NOT NULL, DEFAULT CURRENT_DATE	Tanggal pengembalian
kondisi_saar_kembali	ENUM	NOT NULL, DEFAULT 'baik'	Kondisi barang saat kembali
denda	DECIMAL(12,2)	NOT NULL, DEFAULT 0, CHECK >= 0	Denda keterlambatan
id_petugas	INT	FK -> petugas	Petugas yang memproses pengembalian

9. Tabel audit_log

Nama Field	Tipe Data	Constraint	Keterangan
id_log	INT	PK, AUTO_INCREMENT	Identitas log
nama_tabel	VARCHAR(50)	NOT NULL	Nama tabel yang berubah
aksi	ENUM	NOT NULL	INSERT / UPDATE / DELETE
data_lama	TEXT	-	Data sebelum perubahan
data_baru	TEXT	-	Data setelah perubahan
waktu_aksi	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Waktu perubahan data
user_db	VARCHAR(100)	NOT NULL	User database yang melakukan aksi

2.6 Relasi Antar Tabel

- kategori_barang (1) — (N) barang : satu kategori dapat memiliki banyak barang.
- laboratorium (1) — (N) barang : satu laboratorium dapat menyimpan banyak barang.
- peminjam (1) — (N) peminjaman : satu peminjam dapat melakukan banyak transaksi peminjaman.
- petugas (1) — (N) peminjaman : satu petugas dapat memproses banyak transaksi peminjaman.
- peminjaman (1) — (N) detail_peminjaman : satu transaksi peminjaman dapat berisi banyak item barang.
- barang (1) — (N) detail_peminjaman : satu barang dapat muncul pada banyak detail peminjaman.

- peminjaman (1) — (N) pengembalian : satu transaksi peminjaman berelasi dengan pengembaliannya.
- petugas (1) — (N) pengembalian : satu petugas dapat memproses banyak pengembalian.

Seluruh relasi di atas bertipe one-to-many dan dijaga konsistensinya melalui FOREIGN KEY dengan aturan ON UPDATE CASCADE dan ON DELETE RESTRICT (kecuali detail_peminjaman yang menggunakan ON DELETE CASCADE terhadap peminjaman induknya).

BAB III IMPLEMENTASI DATABASE

3.1 Pembuatan Database dan Tabel

Basis data diberi nama db_inventaris_lab dan terdiri dari 9 tabel utama. Seluruh script pembuatan tabel disimpan pada file 01_schema.sql dan dapat dijalankan ulang dari awal karena menggunakan DROP DATABASE IF EXISTS pada baris pertama. Contoh pembuatan tabel barang beserta foreign key dan constraint-nya:

```
CREATE TABLE barang (  
  id_barang INT AUTO_INCREMENT PRIMARY KEY,  
  nama_barang VARCHAR(100) NOT NULL,  
  id_kategori INT NOT NULL,  
  id_lab INT NOT NULL,  
  stok_total INT NOT NULL DEFAULT 0 CHECK (stok_total >= 0),  
  stok_tersedia INT NOT NULL DEFAULT 0 CHECK (stok_tersedia >= 0),  
  kondisi ENUM('baik','rusak_ringan','rusak_berat') NOT NULL DEFAULT 'baik',  
  harga_satuan DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (harga_satuan >= 0),  
  tanggal_masuk DATE NOT NULL DEFAULT (CURRENT_DATE),  
  CONSTRAINT fk_barang_kategori FOREIGN KEY (id_kategori)  
    REFERENCES kategori_barang(id_kategori)  
    ON UPDATE CASCADE ON DELETE RESTRICT,  
  CONSTRAINT fk_barang_lab FOREIGN KEY (id_lab)  
    REFERENCES laboratorium(id_lab)  
    ON UPDATE CASCADE ON DELETE RESTRICT,  
  CONSTRAINT chk_stok_tersedia_valid CHECK (stok_tersedia <= stok_total)  
) ENGINE=InnoDB;
```

3.2 Implementasi Constraint

Constraint yang diterapkan pada basis data ini meliputi:

- PRIMARY KEY pada setiap tabel (id_kategori, id_lab, id_petugas, id_peminjam, id_barang, id_peminjaman, id_detail, id_pengembalian, id_log).
- FOREIGN KEY untuk menjaga integritas referensial antar tabel, dengan aturan ON UPDATE CASCADE dan ON DELETE RESTRICT/CASCADE sesuai kebutuhan.
- NOT NULL pada kolom yang wajib diisi.
- UNIQUE pada nama_kategori, nama_lab, username petugas, dan no_identitas peminjam.
- CHECK pada stok_total, stok_tersedia, harga_satuan, jumlah, dan denda agar tidak bernilai negatif, serta CHECK tanggal_rencana_kembali >= tanggal_pinjam.
- DEFAULT pada tanggal_masuk, tanggal_pinjam, status, kondisi, dan waktu_aksi.

3.3 Implementasi Stored Procedure

Terdapat 5 Stored Procedure yang diimplementasikan. Berikut contoh sp_pinjam_barang yang menerapkan Transaction Control dan Exception Handling:

```
CREATE PROCEDURE sp_pinjam_barang (  
  IN p_id_peminjam INT, IN p_id_petugas INT, IN p_id_barang INT,  
  IN p_jumlah INT, IN p_tanggal_rencana DATE,  
  OUT p_status_pesan VARCHAR(255)  
)  
BEGIN  
  DECLARE v_stok INT DEFAULT 0;  
  DECLARE v_id_peminjaman INT;  
  DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  BEGIN  
    ROLLBACK;  
    SET p_status_pesan = 'GAGAL: Terjadi kesalahan, transaksi dibatalkan.';
```

```

END;
...
SELECT stok_tersedia INTO v_stok FROM barang WHERE id_barang = p_id_barang;
START TRANSACTION;
IF v_stok < p_jumlah THEN
    ROLLBACK;
    SET p_status_pesan = CONCAT('GAGAL: Stok tidak mencukupi. Stok tersedia hanya ', v_stok, ');
ELSE
    INSERT INTO peminjaman (...) VALUES (...);
    SET v_id_peminjaman = LAST_INSERT_ID();
    INSERT INTO detail_peminjaman (...) VALUES (...);
    COMMIT;
    SET p_status_pesan = CONCAT('BERHASIL: id_peminjaman = ', v_id_peminjaman);
END IF;
END

```

Daftar lengkap Stored Procedure yang dibuat:

No	Nama Procedure	Fungsi
1	sp_pinjam_barang	Mencatat transaksi peminjaman baru dengan validasi stok
2	sp_proses_pengembalian	Memproses pengembalian dan menghitung denda otomatis
3	sp_batalkan_peminjaman	Membatalkan peminjaman dan mengembalikan stok (cursor + savepoint)
4	sp_update_status_terlambat	Memperbarui status peminjaman yang melewati batas waktu (cursor)
5	sp_laporan_peminjaman_periode	Menampilkan laporan transaksi peminjaman berdasarkan periode

3.4 Implementasi Function

Terdapat 3 Function yang diimplementasikan. Berikut contoh fn_hitung_denda:

```

CREATE FUNCTION fn_hitung_denda (p_id_peminjaman INT, p_tanggal_kembali DATE)
RETURNS DECIMAL(12,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_tgl_rencana DATE;
    DECLARE v_hari_telat INT DEFAULT 0;
    DECLARE v_total_barang INT DEFAULT 0;
    DECLARE v_tarif DECIMAL(12,2) DEFAULT 2000.00;

    SELECT tanggal_rencana_kembali INTO v_tgl_rencana
    FROM peminjaman WHERE id_peminjaman = p_id_peminjaman;

    SET v_hari_telat = DATEDIFF(p_tanggal_kembali, v_tgl_rencana);
    IF v_hari_telat < 0 THEN SET v_hari_telat = 0; END IF;

    SELECT IFNULL(SUM(jumlah), 0) INTO v_total_barang
    FROM detail_peminjaman WHERE id_peminjaman = p_id_peminjaman;

    RETURN v_hari_telat * v_tarif * v_total_barang;
END

```

Daftar lengkap Function yang dibuat:

No	Nama Function	Fungsi
1	fn_cek_stok_tersedia(id_barang)	Mengembalikan jumlah stok tersedia suatu barang

No	Nama Function	Fungsi
2	fn_hitung_denda(id_peminjaman, tgl_kembali)	Menghitung denda keterlambatan (Rp 2.000/hari/unit)
3	fn_jumlah_peminjaman_aktif(id_peminjam)	Menghitung jumlah peminjaman aktif seorang peminjam

3.5 Implementasi Trigger

Terdapat 5 Trigger, mencakup validasi data, audit logging, dan perubahan otomatis. Contoh trigger validasi stok sebelum data disimpan:

```
CREATE TRIGGER trg_validasi_stok_before_insert
BEFORE INSERT ON detail_peminjaman
FOR EACH ROW
BEGIN
    DECLARE v_stok INT DEFAULT 0;
    SELECT stok_tersedia INTO v_stok FROM barang WHERE id_barang = NEW.id_barang;

    IF v_stok IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Barang tidak ditemukan.';
    ELSEIF NEW.jumlah > v_stok THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stok barang tidak mencukupi.';
    ELSEIF NEW.jumlah <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Jumlah harus lebih dari 0.';
    END IF;
END
```

Contoh trigger audit logging pada tabel barang:

```
CREATE TRIGGER trg_audit_barang_update
AFTER UPDATE ON barang
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (nama_tabel, aksi, data_lama, data_baru, user_db)
    VALUES ('barang', 'UPDATE',
        CONCAT('id_barang=', OLD.id_barang, ';stok_tersedia=', OLD.stok_tersedia,
            ';kondisi=', OLD.kondisi),
        CONCAT('id_barang=', NEW.id_barang, ';stok_tersedia=', NEW.stok_tersedia,
            ';kondisi=', NEW.kondisi),
        CURRENT_USER());
END
```

Daftar lengkap Trigger yang dibuat:

No	Nama Trigger	Jenis	Fungsi
1	trg_validasi_stok_before_insert	Validasi	Menolak peminjaman jika stok tidak mencukupi
2	trg_kurangi_stok_after_insert	Perubahan otomatis	Mengurangi stok setelah detail peminjaman disimpan
3	trg_tambah_stok_after_pengembalian	Perubahan otomatis	Menambah kembali stok & update status setelah pengembalian
4	trg_audit_barang_update	Audit logging	Mencatat perubahan data barang ke audit_log
5	trg_audit_peminjaman_insert	Audit logging	Mencatat setiap transaksi peminjaman baru ke audit_log

3.6 Implementasi Audit Logging

Audit logging diterapkan melalui tabel `audit_log` yang diisi secara otomatis oleh trigger `trg_audit_barang_update` dan `trg_audit_peminjaman_insert`. Setiap baris `audit_log` mencatat nama tabel yang berubah, jenis aksi, kondisi data sebelum dan sesudah perubahan (dalam bentuk string ringkas), waktu kejadian, dan user database yang melakukan aksi (`CURRENT_USER()`).

3.7 Implementasi Transaction Control

Transaction Control diterapkan pada seluruh Stored Procedure yang mengubah data lebih dari satu tabel. Tiga skenario yang didemonstrasikan:

- 2 Transaksi berhasil (COMMIT): peminjaman barang dengan stok mencukupi diproses hingga selesai dan disimpan permanen.
- 3 Transaksi gagal (ROLLBACK): peminjaman dengan jumlah melebihi stok tersedia dibatalkan seluruhnya sehingga data peminjaman maupun stok tidak berubah.
- 4 Transaksi sebagian dibatalkan (SAVEPOINT): pada `sp_batalkan_peminjaman`, setiap item barang dikembalikan stoknya menggunakan SAVEPOINT per item. Jika satu item gagal diproses, hanya perubahan pada item tersebut yang di-ROLLBACK TO SAVEPOINT, sedangkan item lain tetap diproses.

3.8 Implementasi Cursor

Cursor digunakan pada dua Stored Procedure. Pada `sp_update_status_terlambat`, cursor membaca seluruh transaksi berstatus 'dipinjam' satu per satu dan memperbarui menjadi 'terlambat' apabila tanggal_rencana_kembali telah lewat:

```
DECLARE cur_terlambat CURSOR FOR
  SELECT id_peminjaman, tanggal_rencana_kembali
  FROM peminjaman WHERE status = 'dipinjam';
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

OPEN cur_terlambat;
cek_loop: LOOP
  FETCH cur_terlambat INTO v_id_peminjaman, v_tgl_rencana;
  IF v_done = 1 THEN LEAVE cek_loop; END IF;
  IF v_tgl_rencana < CURRENT_DATE THEN
    UPDATE peminjaman SET status = 'terlambat' WHERE id_peminjaman = v_id_peminjaman;
    SET v_counter = v_counter + 1;
  END IF;
END LOOP;
CLOSE cur_terlambat;
```

Cursor kedua digunakan pada `sp_batalkan_peminjaman` untuk memproses setiap baris detail_peminjaman satu per satu sambil mengembalikan stok masing-masing barang.

3.9 Implementasi Indexing

Index dibuat pada kolom yang sering dijadikan kondisi pencarian namun belum memiliki index otomatis. Perlu dicatat bahwa MySQL/MariaDB (InnoDB) secara otomatis membuat index pada kolom FOREIGN KEY, sehingga index tambahan dibuat pada kolom `peminjaman.tanggal_pinjam` dan `barang.nama_barang` agar perbandingan sebelum-sesudah index lebih representatif.

```
CREATE INDEX idx_peminjaman_tanggal ON peminjaman(tanggal_pinjam);
CREATE INDEX idx_barang_nama ON barang(nama_barang);
```

Analisis lengkap hasil EXPLAIN sebelum dan sesudah index dibahas pada Bab IV Pengujian.

BAB IV PENGUJIAN

Pengujian dilakukan dengan menjalankan seluruh script SQL (01_schema.sql s.d. 06_demo_pengujian.sql) secara berurutan pada MariaDB melalui XAMPP. Seluruh script terbukti dapat dijalankan ulang dari awal tanpa error.

4.1 Pengujian Insert, Update, Delete

Pengujian INSERT dilakukan pada seluruh tabel master (kategori_barang, laboratorium, petugas, pinjaman, barang) dengan data dummy dan berhasil tersimpan sesuai constraint yang ditetapkan. Pengujian UPDATE dilakukan pada tabel laboratorium (mengubah penanggung_jawab) dan tabel barang (mengubah kondisi), yang keduanya berhasil dan pada tabel barang turut memicu trigger audit logging. Pengujian DELETE pada tabel kategori_barang yang masih direferensikan oleh tabel barang ditolak oleh sistem karena FOREIGN KEY ON DELETE RESTRICT, sesuai dengan tujuan menjaga integritas data.

4.2 Pengujian Stored Procedure

Pemanggilan CALL sp_pinjam_barang(1, 1, 1, 2, '2026-07-21', @pesan) untuk peminjaman 2 unit Multimeter Digital menghasilkan:

```
hasil_pinjam_1: BERHASIL: Peminjaman tercatat dengan id_peminjaman = 1
stok_multimeter_setelah_pinjam: 18 (semula 20, berkurang 2 sesuai jumlah pinjam)
Prosedur sp_proses_pengembalian yang dipanggil pada hari yang sama dengan tanggal rencana kembali menghasilkan denda Rp 0 karena belum melewati batas waktu, dan status peminjaman otomatis berubah menjadi 'dikembalikan'.
```

4.3 Pengujian Function

Function fn_cek_stok_tersedia(1) mengembalikan nilai 18 setelah peminjaman pertama, sesuai dengan pengurangan stok oleh trigger. Function fn_jumlah_peminjaman_aktif(1) mengembalikan nilai 1 saat peminjam memiliki satu transaksi berstatus 'dipinjam'. Function fn_hitung_denda diuji melalui sp_proses_pengembalian dan menghasilkan nilai yang sesuai dengan jumlah hari keterlambatan dikalikan tarif dan jumlah barang.

4.4 Pengujian Trigger

Trigger trg_validasi_stok_before_insert diuji dengan mencoba meminjam barang melebihi stok tersedia melalui sp_pinjam_barang (999 unit Osiloskop, stok hanya 5 unit). Hasil pengujian:

```
skenario_gagal_rollback: GAGAL: Stok tidak mencukupi. Stok tersedia hanya 5.
stok_tersedia (setelah percobaan): 5 (tidak berubah)
jumlah_peminjaman_seharusnya_tidak_bertambah: 0
Trigger trg_audit_barang_update diuji dengan mengubah kondisi barang id_barang = 5 dari 'baik' menjadi 'rusak_ringan'. Baris baru pada audit_log tercatat sebagai berikut:
```

```
nama_tabel : barang
aksi      : UPDATE
data_lama  : id_barang=5;nama_barang=Proyektor;stok_tersedia=6;kondisi=baik
data_baru  : id_barang=5;nama_barang=Proyektor;stok_tersedia=6;kondisi=rusak_ringan
user_db    : root@localhost
```

Trigger trg_audit_peminjaman_insert terbukti mencatat setiap transaksi peminjaman baru (id_peminjaman 1, 2, dan 3) ke dalam audit_log secara otomatis tanpa perlu perintah INSERT tambahan dari aplikasi.

4.5 Pengujian Audit Log

Tabel audit_log terisi otomatis setiap kali trigger audit dijalankan, mencakup data_lama, data_baru, waktu_aksi, dan user_db. Hal ini membuktikan sistem mampu merekam jejak aktivitas perubahan data tanpa campur tangan manual.

4.6 Pengujian Transaction Control

Skenario	Aksi Pengujian	Hasil
COMMIT	Meminjam 3 unit Gelas Ukur (stok tersedia 50)	BERHASIL, stok_tersedia berkurang menjadi 47 dan tersimpan permanen
ROLLBACK	Meminjam 999 unit Osiloskop (stok tersedia 5)	GAGAL, seluruh transaksi dibatalkan, stok tetap 5
SAVEPOINT	Membatalkan peminjaman Mikroskop menggunakan sp_batalkan_peminjaman	BERHASIL, stok_tersedia kembali bertambah, 0 item gagal diproses

4.7 Pengujian Exception Handling

Skenario Error	Pesan yang Dihasilkan Sistem
Data peminjam tidak ditemukan (id_peminjam = 9999)	GAGAL: Data peminjam tidak ditemukan.
Jumlah pinjam bernilai negatif (-5)	GAGAL: Jumlah pinjaman harus lebih dari 0.

Kedua skenario di atas membuktikan bahwa exception handling pada Stored Procedure (DECLARE EXIT HANDLER FOR SQLEXCEPTION) dan validasi logis berhasil mencegah data tidak valid tersimpan ke database.

4.8 Pengujian Indexing

Sebelum index dibuat, EXPLAIN terhadap pencarian berdasarkan tanggal_pinjam dan nama_barang menunjukkan:

SEBELUM INDEX

Tabel peminjaman : type=ALL, key=NULL, rows diperiksa=seluruh baris tabel

Tabel barang : type=ALL, key=NULL, rows diperiksa=seluruh baris tabel (Using where)

Setelah index idx_peminjaman_tanggal dan idx_barang_nama dibuat, EXPLAIN terhadap query yang sama menunjukkan:

SETELAH INDEX

Tabel peminjaman : type=ref, key=idx_peminjaman_tanggal, rows diperiksa=1 baris (Using index condition)

Tabel barang : type=ref, key=idx_barang_nama, rows diperiksa=1 baris (Using index condition)

Kesimpulan pengujian: sebelum index dibuat, MySQL melakukan full table scan (type = ALL) sehingga seluruh baris tabel harus diperiksa satu per satu. Setelah index dibuat, optimizer memanfaatkan index yang tersedia (type = ref) sehingga jumlah baris yang diperiksa berkurang signifikan menjadi hanya baris yang relevan. Pada tabel dengan jumlah data yang jauh lebih besar dari data dummy pengujian ini, perbedaan performa akan semakin signifikan karena full table scan bertumbuh linear terhadap jumlah baris, sedangkan pencarian melalui index bertumbuh logaritmik (B-Tree).

4.9 Hasil dan Pembahasan

Secara keseluruhan, seluruh komponen wajib pada spesifikasi proyek (tabel dan relasi, Stored Procedure, Function, Trigger, Audit Logging, Transaction Control, Exception Handling, Cursor, dan Indexing) telah berhasil diimplementasikan dan diuji. Sistem mampu menjaga konsistensi stok barang secara otomatis melalui trigger, mencegah transaksi yang tidak valid melalui exception handling, membatalkan transaksi secara aman melalui rollback dan savepoint, serta merekam jejak audit setiap perubahan data penting.

BAB V PENUTUP

5.1 Kesimpulan

1. Sistem Basis Data Inventaris Laboratorium berhasil dirancang dengan 9 tabel utama yang saling berelasi one-to-many, dilengkapi primary key, foreign key, dan constraint yang menjaga validitas data.
2. Logika bisnis utama seperti peminjaman, pengembalian, pembatalan transaksi, dan pelaporan berhasil diimplementasikan melalui 5 Stored Procedure yang saling terintegrasi dengan Function dan Trigger.
3. Trigger yang diimplementasikan terbukti mampu melakukan validasi data secara otomatis, mencatat audit log setiap perubahan data penting, serta memperbarui stok barang secara otomatis tanpa campur tangan aplikasi.
4. Transaction Control (COMMIT, ROLLBACK, SAVEPOINT) dan Exception Handling terbukti menjaga konsistensi data pada saat terjadi kegagalan proses, seperti stok tidak mencukupi atau data tidak ditemukan.
5. Cursor berhasil diterapkan untuk memproses data lebih dari satu baris, yaitu pada pembaruan status keterlambatan massal dan pengembalian stok saat pembatalan transaksi.
6. Indexing pada kolom tanggal_pinjam dan nama_barang terbukti mengubah strategi eksekusi query dari full table scan (type=ALL) menjadi pencarian berbasis index (type=ref), yang akan meningkatkan efisiensi secara signifikan pada volume data yang lebih besar.

5.2 Saran Pengembangan

- Menambahkan aplikasi antarmuka (web/desktop) agar sistem dapat digunakan langsung oleh petugas laboran dan peminjam tanpa mengakses database secara langsung.
- Menambahkan fitur notifikasi otomatis (mis. email/WhatsApp) kepada peminjam yang mendekati atau melewati batas waktu pengembalian.
- Menambahkan role-based access control pada tabel petugas agar hak akses setiap petugas dapat dibedakan sesuai jabatan.
- Melakukan uji performa indexing dengan volume data yang lebih besar (ribuan hingga jutaan baris) untuk mengukur peningkatan performa secara kuantitatif.
- Menambahkan mekanisme backup dan recovery basis data secara berkala untuk menjaga keamanan data inventaris laboratorium.